Class 6: Container Orchestration with Kubernetes

Session Overview

- **Understand the concepts of DevOps** and the importance of container orchestration.
- **Learn about Kubernetes** as a leading container orchestration tool.
- **Get hands-on experience** with deploying and managing containerized applications in Kubernetes.

Key Topics:

- Introduction to Kubernetes
- Understanding Pods, Services, and Deployments
- Managing applications with Kubernetes
- Hands-on with Minikube (or another Kubernetes environment)

Prerequisites:

- Familiarity with **containerization** (Docker).
- Basic understanding of **DevOps practices** like Continuous Integration and Continuous Delivery (CI/CD).

Introduction to Kubernetes

Kubernetes is an open-source container orchestration platform designed to automate deploying, scaling, and managing containerized applications. It groups containers into logical units, making it easier to manage and scale applications seamlessly.

Why Kubernetes?

- **Scalability**: Automatically scales your application based on the traffic load.
- **Self-healing**: If containers fail, Kubernetes automatically restarts or replaces them.
- **Declarative Configuration**: Describes your infrastructure and application state using simple YAML or JSON files.
- **Multi-cloud and hybrid support**: Works across various environments (on-premises, cloud, hybrid).

Core Concepts in Kubernetes

1. **Node**
    - A physical or virtual machine that runs your application workloads.
    - Contains the components to run containers and communicate with the Kubernetes control plane.
    - Managed by the **Kubernetes master (control plane)**.
2. **Pod**
    - The smallest and simplest Kubernetes object.
    - A pod can run one or more containers.
    - All containers in a pod share the same network namespace and storage volumes.
3. **Service**
    - A stable endpoint to access a group of pods.
    - It provides load balancing and helps expose your application internally or externally.
4. **Deployment**
    - Manages the deployment and scaling of pods.
    - Ensures that the desired number of pod replicas are running.
5. **Namespace**
    - A logical isolation boundary within a Kubernetes cluster to separate different environments or teams.
6. **Kubelet**
    - A component that runs on each node, ensuring that the containers are running in a pod.

Kubernetes Architecture

Kubernetes architecture consists of two main components:

1. **Control Plane (Master Node)**:
    - **API Server**: Exposes Kubernetes APIs for interaction (kubectl).
    - **Scheduler**: Assigns new pods to nodes.
    - **Controller Manager**: Handles node failures, pod replication, etc.
    - **etcd**: A distributed key-value store to store configuration data.
2. **Worker Nodes**:
    - **Kubelet**: Communicates with the API server and ensures containers are running.
    - **Kube Proxy**: Manages networking rules on nodes to ensure communication between pods.
    - **Container Runtime**: Software like Docker or containerd that runs your containers.

Hands-on with Kubernetes

To understand Kubernetes practically, let’s deploy a simple application.

Kubernetes core concepts

Kubernetes provides a robust framework to manage containerized applications. The key components and the control plane work together to ensure that applications are highly available, scalable, and self-healing.

1. Key Kubernetes Components

Pods

- **Definition**: A pod is the smallest and simplest unit in Kubernetes. It encapsulates one or more containers that share the same storage, networking, and specifications.
- **Function**: Pods ensure that containers in the same pod can communicate with each other efficiently, and they share a network namespace.
- **Usage**: Usually, one container is run per pod, but multiple containers can be deployed if they need to work closely together (like a main app and helper app).

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: my-container
      image: nginx
      ports:
        - containerPort: 80
```

Deployments

- **Definition**: A deployment is a higher-level Kubernetes resource that manages pod lifecycles, ensuring the right number of pods are running.
- **Function**: It allows you to scale your application by increasing or decreasing the number of pods. It also supports rolling updates, enabling seamless updates to new versions without downtime.
- **Usage**: You define a deployment with specifications for pod replicas, the container image, and the update strategy.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
```

```yaml
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-container
          image: nginx:1.19.0
          ports:
            - containerPort: 80
```

Services

- **Definition**: A service is an abstraction that defines a logical set of pods and a policy to access them.
- **Function**: It provides load balancing and ensures that applications running in pods can be accessed internally (within the cluster) or externally (from outside the cluster).
- **Types of Services**:
    - **ClusterIP**: Exposes the service on an internal IP in the cluster.
    - **NodePort**: Exposes the service on a static port on each node.
    - **LoadBalancer**: Exposes the service externally using a cloud provider's load balancer.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
```

```
    targetPort: 80
  type: ClusterIP
```

```
apiVersion: v1
kind: Service
metadata:
  name: my-service-nodeport
spec:
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30007
  type: NodePort
```

```
apiVersion: v1
kind: Service
metadata:
  name: my-service-loadbalancer
spec:
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: LoadBalancer
```

ConfigMaps

- **Definition**: A ConfigMap is a Kubernetes object that allows you to store configuration data in key-value pairs.
- **Function**: It decouples application configuration from container images, making it easy to change configurations without rebuilding container images.

- **Usage**: ConfigMaps can be injected into pods as environment variables or mounted as configuration files.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-configmap
data:
  app.properties: |
    database=postgres
    cache.enabled=true
```

Secrets

- **Definition**: Secrets are similar to ConfigMaps but specifically designed for sensitive information like passwords, tokens, or SSH keys.
- **Function**: Kubernetes stores secrets in an encoded format and passes them securely to pods.
- **Usage**: Secrets can be used as environment variables, or they can be mounted as files inside the pod.

```
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
data:
  username: YWRtaW4=
  password: MWYyZDFlMmU=
```

ReplicaSets

- **Definition**: A ReplicaSet ensures that a specified number of pod replicas are running at all times.

- **Function**: It automatically replaces failed pods to maintain the desired state.
- **Usage**: It is usually managed by a deployment, which simplifies its use, but it can also be used directly for more control over pod replica management.

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: my-replicaset
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-container
          image: nginx
          ports:
            - containerPort: 80
```

StatefulSets

- **Definition**: StatefulSets are a specialized Kubernetes resource for managing stateful applications.
- **Function**: Unlike Deployments and ReplicaSets, StatefulSets provide guarantees about the ordering and uniqueness of pods. They are useful for applications that require stable, unique network identifiers, persistent storage, or ordered deployment and scaling.
- **Usage**: It's ideal for workloads like databases, where pod identity and storage persistence are critical.

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: my-statefulset
spec:
  serviceName: "my-service"
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-container
          image: nginx
          volumeMounts:
            - name: my-storage
              mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
    - metadata:
        name: my-storage
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi
```

2. Kubernetes Control Plane and Components

The **Control Plane** is responsible for managing the overall state of the cluster, ensuring that the desired state (as declared by users) is maintained. It consists of several key components:

API Server

- **Definition**: The API Server is the front end of the Kubernetes control plane. It exposes the Kubernetes API, which is used by the `kubectl` command-line tool and other components to interact with the cluster.
- **Function**: The API Server processes and validates requests and updates the state of objects like Pods, Services, and Deployments in etcd (a key-value store).
- **Role**: It's the main point of communication between the control plane and the nodes.

Scheduler

- **Definition**: The Scheduler is responsible for assigning newly created pods to nodes based on resource requirements and policies.
- **Function**: It ensures that the pods are placed on nodes that have sufficient resources (CPU, memory) and adhere to scheduling policies like affinity or anti-affinity.
- **Role**: It keeps the cluster balanced by distributing workloads across nodes.

Controller Manager

- **Definition**: The Controller Manager is a daemon that runs various controllers to ensure that the cluster state matches the desired state defined in Kubernetes configurations.
- **Function**: It monitors the cluster, and when it detects discrepancies (e.g., missing pod replicas), it takes corrective actions (e.g., creates new pods).
- **Key Controllers**:
    - **Node Controller**: Manages the health of nodes.
    - **Replication Controller**: Ensures the desired number of pod replicas are running.
    - **Endpoint Controller**: Updates the endpoints of services.

etcd

- **Definition**: etcd is a distributed, reliable key-value store that stores all cluster data.
- **Function**: It holds the entire state of the cluster, including information about nodes, pods, services, and configuration.
- **Role**: It’s the source of truth for Kubernetes, and any changes to the cluster state are reflected in etcd.

Setting up a Kubernetes Cluster

There are multiple ways to set up a Kubernetes environment, depending on the scale, purpose, and available resources. Here are the most popular options:

1. Minikube

- **Overview**: Minikube is a lightweight Kubernetes implementation that runs a single-node Kubernetes cluster on your local machine. It is ideal for development and testing purposes.
- **Key Features**:
    - Simple setup and installation.
    - Allows testing of Kubernetes features on a local environment.
    - Supports various container runtimes (Docker, containerd, etc.).
- **Use Case**: Best for developers who want to test Kubernetes locally before moving to a production environment.

2. K3s

- **Overview**: K3s is a lightweight Kubernetes distribution designed for IoT and edge computing but can also be used for local development.
- **Key Features**:
    - Lightweight and fast.
    - Simple to install and deploy on low-resource environments.
    - Optimized for single-node or small clusters.
- **Use Case**: Perfect for edge computing or development setups with limited resources.

3. Cloud Providers

- **Overview**: Major cloud providers offer fully managed Kubernetes services, which eliminate the need to manually set up and manage Kubernetes clusters.
- **Popular Providers**:
    - **Google Kubernetes Engine (GKE)**: Managed Kubernetes offering by Google Cloud.
    - **Amazon Elastic Kubernetes Service (EKS)**: Managed Kubernetes on AWS.
    - **Azure Kubernetes Service (AKS)**: Kubernetes service on Microsoft Azure.
- **Key Features**:
    - Fully managed, with the control plane maintained by the cloud provider.
    - Integration with other cloud services for networking, storage, and scaling.
    - Automated updates, backups, and monitoring.
- **Use Case**: Suitable for production-level applications, large-scale deployments, and enterprises seeking scalability and resilience.

4. kubeadm

- **Overview**: kubeadm is a tool that provides best-practice fast setup for Kubernetes clusters on existing infrastructure (e.g., on-prem servers or virtual machines).
- **Key Features**:

    - Deploys multi-node Kubernetes clusters.
    - Gives more control over the setup than managed services or Minikube.
    - Ideal for advanced users who want to manage their own Kubernetes infrastructure.
- **Use Case**: Ideal for on-premise setups or when more control over the cluster setup and configuration is required.

Step-by-Step Guide to Setting Up a Basic Kubernetes Cluster

Step 1: Install Kubernetes (Minikube for Local Testing)

Minikube is a tool that sets up a local Kubernetes cluster for learning.

```
# Install Minikube (if not installed already)
curl -LO
https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd6
4
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Start Minikube cluster
minikube start

# Install kubectl
sudo apt-get install -y kubectl
```

Step 2: Create a Deployment

Deploy an application (e.g., Nginx) using a **Kubernetes Deployment**.

```
# Create a deployment
kubectl create deployment nginx --image=nginx

# Check deployment status
kubectl get deployments
```

```
# Check the running pods
kubectl get pods
```

Step 3: Expose the Deployment as a Service

Expose the application to make it accessible outside the cluster.

```
# Expose the deployment
kubectl expose deployment nginx --port=80 --type=NodePort

# Get the service details
kubectl get services
```

Step 4: Access the Application

Access your application using Minikube's IP and the node port assigned.

```
# Get the Minikube IP
minikube ip

# Open the application
minikube service nginx
```

Step 5: Scale the Deployment

Increase the number of replicas to handle more traffic.

```
# Scale the deployment to 3 replicas
kubectl scale deployment nginx --replicas=3

# Verify the number of pods
kubectl get pods
```

Step 6: Clean Up Resources

After testing, you can clean up by deleting the deployment and service.

```
# Delete the service
kubectl delete service nginx

# Delete the deployment
kubectl delete deployment nginx
```